# THE SIDEBAR IS A SECURITY BOUNDARY
# UI-AS-POLICY FOR CAPABILITY-DRIVEN MULTI-TENANT SAAS

Abstract. Multi-tenant SaaS products tend to develop a particular kind of split-brain: the backend enforces permissions, while the frontend *guesses* what the user should be able to see. Over time, those guesses drift. The UI becomes a polite liar (buttons that 403) and occasionally a dangerous liar (surfaces that imply access that should never exist).

We present UI-as-Policy: a capability-driven architecture where the UI is treated as a first-class policy surface. The central idea is UI-as-Data: the backend materializes a tenant- and role-scoped *UI contract* (enabled modules and structured layout fragments) and delivers it at runtime. The web tier then composes navigation and module experiences from this contract while deriving an identical authorization model server-side and exporting it to the client in a packed form for UX consistency.

Our production implementation couples (i) a control plane API (AWS Lambda) that returns `/v2/me` with an explicit *actor* vs. *`view-as`* split, (ii) tenant policy stored as data in DynamoDB (modules and layout fragments), and (iii) server-authoritative abilities (CASL) serialized for the client. This design reduces tenant onboarding friction (configuration over redeploy), minimizes frontend/backend authorization drift, and enables safe support impersonation without sacrificing auditability.

## Contents



## 1. Introduction

A multi-tenant product makes promises. Every menu item, route, and page is a promise that says: "if you click this, something meaningful will happen." When those promises are not backed by policy, the UI turns into a museum of broken doors: very interactive, not very useful.

The canonical failure mode is subtle. The backend is (usually) correct: it checks authorization and rejects requests. The frontend, however, often carries its own parallel permission model—a collage of route guards, role checks, and hardcoded navigation conditions. These two models drift as the product grows. Users see actions that fail at runtime; teams ship patchy fixes; and eventually someone adds a "temporary" admin override that becomes permanent.

We adopt a simple principle:

*Date*: 2026-03-23.

**The UI is a security boundary. If the UI can misrepresent access, it is part of the authorization problem.**

This paper describes how Causify Platform implements UI-as-Policy using UI-as-Data. The backend produces a tenant-scoped *UI contract* consisting of: (i) the set of enabled modules for the effective tenant/role, and (ii) structured layout fragments that drive navigation and module configuration. The web tier renders a capability-driven interface composed from these fragments and guarded by the same authorization semantics derived server-side.

What is UI-as-Policy? UI-as-Policy is not "authorization in the frontend." The backend remains the enforcement point. UI-as-Policy means the UI is derived from the same policy inputs as authorization, so the user experience does not drift away from what the system will actually permit.

Contributions.

- **UI-as-Data as a UI contract:** a data model and delivery pattern where tenants and roles receive an explicit, runtime UI contract (modules + layout fragments), rather than a hardcoded UI with scattered conditional logic.
- **Server-authoritative abilities exported for UX:** a single source of truth for permissions built on the server and packed for client-side UX (CASL), minimizing frontend/backend divergence [5].
- **Auditable `view-as` semantics:** an explicit separation between *actor* identity and *effective* tenant/role context, enabling safe support/QA workflows without shared admin accounts.
- **Concrete implementation:** an end-to-end architecture using Cognito, Lambda (Powertools), DynamoDB, and Next.js [1–3, 12].

## 2. Problem Statement and Motivation

A tenant-aware product must satisfy requirements that are individually straightforward but jointly annoying:

- **R1 (Backend correctness):** authorization must be enforced on the server and remain non-bypassable.
- **R2 (UX truthfulness):** the UI should not advertise actions the user cannot perform. (No broken doors.)
- **R3 (Tenant variability):** tenants differ by enabled modules, default views, and configuration. Roles differ within a tenant.
- **R4 (Low operational overhead):** onboarding or changing a tenant should not require code forks or redeploys.
- **R5 (Support at scale):** operators need safe mechanisms to reproduce tenant issues (e.g., impersonation) while preserving auditability.

Traditional RBAC can handle coarse role-based access, but often struggles when permissions must vary across tenants and evolve rapidly. ABAC provides a more expressive foundation [8], but it does not automatically solve the *UI drift problem*: even a perfectly designed backend policy can still leave the frontend guessing.

A common alternative is feature flags: ship everything and selectively enable parts [7]. Feature flags help decouple rollout from deployment, but they tend to accumulate into a second configuration universe: one system decides what exists, another decides what is visible, and neither system owns the UX contract.

| Capability | Traditional | UI-as-Policy |
|---|---|---|
| Backend enforcement | ✓ | ✓ |
| Role-based access | ✓ | ✓ |
| Tenant isolation | ✓ | ✓ |
| Unified policy | × | ✓ |
| Derived UI | × | ✓ |
| Auto-sync | × | ✓ |
| Module independence | × | ✓ |
| Config-only tenants | × | ✓ |
| Zero redeploy | × | ✓ |
| UI truthfulness | × | ✓ |

**Table 1.** Capability comparison: traditional authorization versus UI-as-Policy. Both approaches provide core security; UI-as-Policy adds consistency and operational efficiency.

Motivation. We want a system where: (i) tenant/role variability is expressed as data, (ii) the backend can materialize an authoritative view of "what this user sees", (iii) the frontend consumes that view to compose navigation and defaults, and (iv) authorization semantics remain consistent across backend and frontend. The motivation is not to make the UI dynamic for its own sake, but to make it *honest*.

## 3. System Overview

This section introduces the end-to-end flow and the artifacts exchanged between components. The key is that the backend produces a *materialized* UI contract scoped by tenant and role, and the frontend renders it without inventing policy.

3.1. **High-level architecture.** Figure 1 shows the main components and control flow.

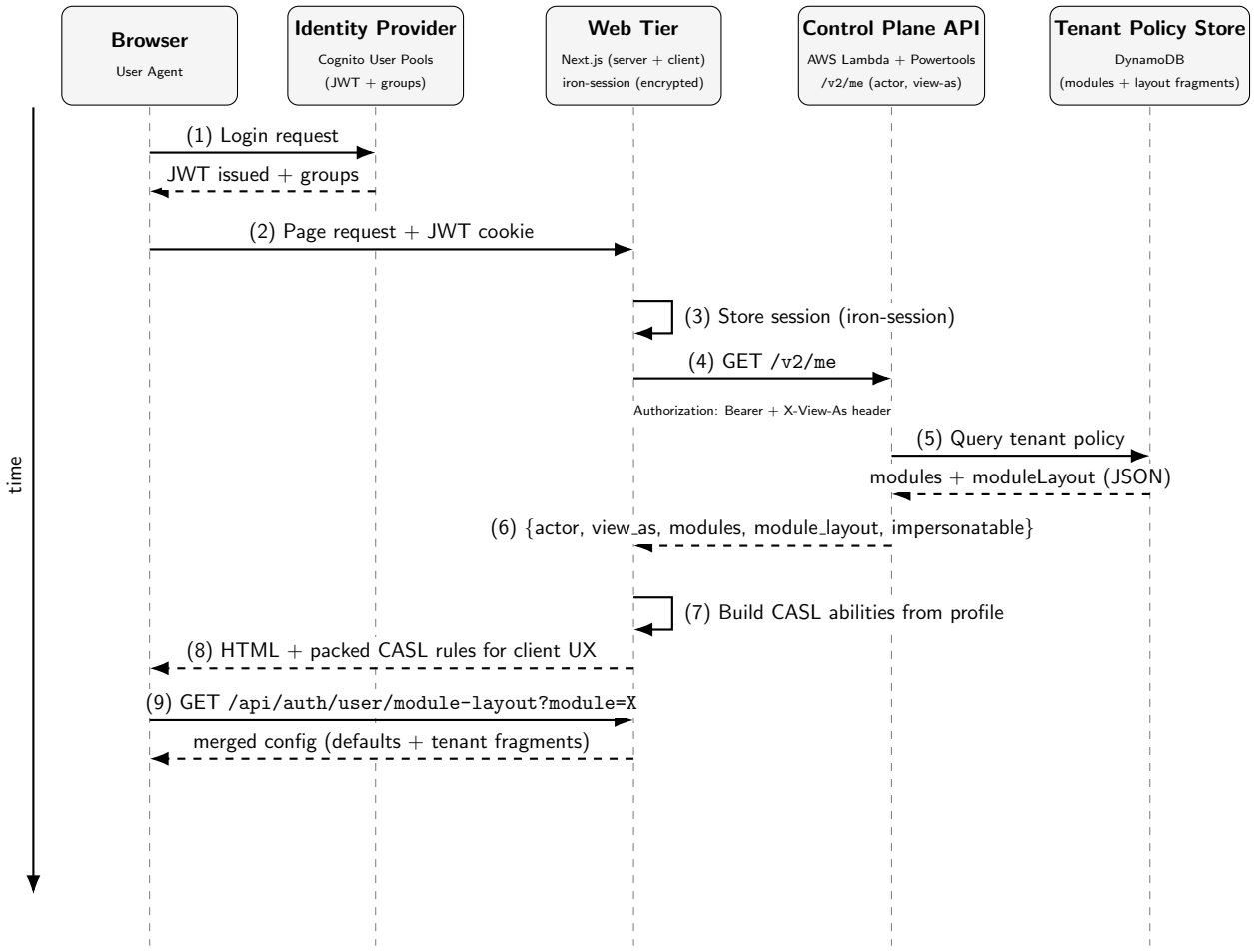


**Figure 1.** Causify Platform sequence diagram showing the complete request lifecycle. (1–2) Browser authenticates via Cognito and receives JWT with group claims. (3–4) Web Tier stores session and fetches materialized profile from Control Plane. (5–6) Control Plane queries DynamoDB for tenant policy and returns the UI contract. (7–8) Server builds CASL abilities and returns HTML with packed rules. (9) Module layouts are lazy-loaded on demand.

3.2. **Request lifecycle (what happens on a typical page load).** The core request lifecycle is:

(1) **Authentication:** a user authenticates via Cognito and the web tier receives a JWT that includes group membership [1, 2].

(2) **Control-plane materialization (`/v2/me`):** the web tier calls the control plane with `Authorization: Bearer ...` and optionally `X-View-As: tenant_id=...;role=...`. The Lambda middleware validates the token, computes `user_role`, computes an

`impersonatable` allow-list, validates `view-as` selections against that allow-list, and returns:

- `actor`: immutable identity (who is calling),
- `view_as`: effective tenant/role and derived modules/layout,
- `impersonatable`: permissible `view-as` targets for support/QA.

The response includes caching and ETag support to reduce repeated materialization overhead (useful when the UI pings "who am I?" often).

(3) **Server-authoritative abilities:** the web tier builds the canonical permission model from the server-side session profile and exports packed rules for the client (CASL) [5]. The client uses these rules for UX (what to show), while backend APIs remain authoritative for security (what is allowed).

(4) **Runtime UI composition:** the UI composes navigation and module defaults from `module_layout` returned by `/v2/me`. Module-specific layout is lazily loaded and merged with shipped defaults, so configuration can be partial (override the sidebar, not the universe).

3.3. **Key abstractions and artifacts.** Table 2 summarizes the artifacts that make the system composable and auditable.

| Artifact | Where defined | Produced by | Used for |
|---|---|---|---|
| `actor` | JWT + derived claims | `/v2/me` (Lambda) | Auditability: the real caller identity never changes. |
| `view_as` | `X-View-As` header | `/v2/me` (validated) | Effective tenant/role context for modules + layout. |
| `impersonatable` | Derived from actor role | `/v2/me` | Guardrail: explicit allow-list of `view-as` targets. |
| `modules` | Tenant policy record | `/v2/me` | Capability list for UI composition and ability building. |
| `module_layout` | Tenant policy (JSON) | `/v2/me` | Structured UI contract: navigation + module defaults. |
| Packed CASL rules | Ability builder | `/api/.../ability` | Client UX consistency: hide forbidden UI. |

**Table 2.** The system's core artifacts: identity, effective context, capability set, and UI contract. The same policy inputs drive both authorization and UX.

3.4. **Why this is different from "dynamic UI".** Many systems have tenant-specific UIs. The distinguishing property here is not mere variability; it is *policy alignment*. UI-as-Policy ensures that what the user sees is derived from the same tenant/role-scoped policy that determines what the system will allow. In short: the UI stops improvising.

3.5. **Module architecture: independence with shared policy.** Causify Platform implements a modular architecture where each functional area operates as an *independent module*. Every module follows an identical structural pattern:

- **Server layout** (`layout.tsx`): enforces CASL access before rendering.
- **Client component** (`client.tsx`): interactive UI, state management, and data fetching.
- **Dedicated API routes** (`/api/{module}/`): inherit the same CASL and `view-as` context.

This yields a key property: modules are *independently evolvable* and can be enabled or disabled per tenant via policy without creating dead routes or broken doors. Listing 1 illustrates the protection pattern.

```tsx
// app/(protected)/(app)/{module}/layout.tsx
import { requireAccess } from '@/lib/core/casl/server'

export default async function ModuleLayout({
  children
}: { children: React.ReactNode }) {
  await requireAccess('module-name')
  return <>{children}</>
}
```

**Listing 1.** Module protection pattern: server layout enforces CASL before rendering.

## 4. Policy Model: From Tenants to Capabilities

This section specifies the Policy-as-Data model that powers UI-as-Policy. The key design choice is to store tenant experience as data and *materialize it server-side* into a "UI contract" that the web tier can consume without inventing permissions in the browser.

4.1. **Identity claims and role derivation.** Authentication is delegated to Amazon Cognito user pools [1]. The control plane receives an ID token and derives two primitives:

- **Actor tenant:** extracted from a custom JWT claim (e.g., `custom:tenantId`).
- **Actor role:** derived from `cognito:groups` [2].

Groups follow a simple convention:

$$\texttt{tenant\_id:role} \quad \text{(e.g., } \texttt{acme:member}, \texttt{causify:admin}\text{)}$$

Users may belong to multiple groups for a tenant. We resolve role by precedence:

$$\texttt{admin} > \texttt{member} > \texttt{demo}$$

This rule is intentionally boring. Boring is good. Boring prevents the "surprise admin" incident that ruins everyone's weekend.

**Algorithm 1** Role resolution from Cognito groups (simplified)

**Require:** tenant_id, groups
1: **if** groups is empty **then return** `None`
2: **end if**
3: valid ← { g ∈ groups : `":"` ∈ *g* and g startsWith `tenant_id:` }
4: **if** valid is empty **then return** `None`
5: **end if**
6: roles ← extract suffix after `":"` for each g in valid
7: order ← [`admin`, `member`, `demo`]
8: **return** argmin_r index(r in order) (default to end if unknown)

4.2. **Impersonation policy: who may `view-as` whom.** UI-as-Policy includes a first-class `view-as` mechanism to support operations, QA, and demo flows without shared admin credentials. This is also the fastest way to reproduce tenant-specific UI bugs: you *become* the bug (politely).

We compute an explicit `impersonatable` allow-list in the control plane:

- **Super admin:** a designated tenant/role pair may impersonate all active tenants and roles (enumerated from the tenant registry table).
- **Tenant admin:** an admin may impersonate all roles *within* their tenant.
- **Everyone else:** may impersonate only what their group memberships allow (e.g., if they are in `acme:demo` and `acme:member`, they may `view-as` either).

This allow-list is returned to the web tier, enabling the UI to offer safe tenant/role selection while keeping enforcement server-side.

4.3. **Tenant policy as data (DynamoDB).** Tenant variability is expressed as Policy-as-Data: a DynamoDB record keyed by `tenant_id`. The record contains tenant metadata plus a role map under `group`. Each role entry provides:

- `modules`: the enabled module identifiers for that role.
- `moduleLayout`: a map of JSON *strings* that represent UI layout/config fragments. These fragments are parsed server-side during materialization.

```
{
  "tenant_id": "acme",
  "domain": "acme.example",
  "status": "active",
  "allowed": true,
  "schema_prefix": "acme",
  "group": {
    "demo": {
      "modules": ["horizon"],
      "moduleLayout": {
        "common": "{ \"sideBar\": { \"defaultCollapsed\": true } }"
      }
    },
    "member": {
      "modules": ["horizon", "grid"],
      "moduleLayout": {
        "common": "{ \"sideBar\": { \"defaultCollapsed\": false }, \"feedback\": { \"enabled
    \": true } }",
        "grid":   "{ \"sidebar\": { \"enabled\": true }, \"table\": { \"pageSize\": 50 } }"
      }
    },
    "admin": {
      "modules": ["horizon", "grid", "datamap"],
      "moduleLayout": {
        "common": "{ \"settings\": { \"enabled\": true }, \"sideBar\": { \"showAdmin\": true }
     }"
      }
    }
  }
}
```

**Listing 2.** Illustrative tenant policy record (close to production shape). Role entries live under `group`. Layout fragments are stored as JSON strings and parsed server-side during materialization into structured objects.

Validation and failure behavior. Because this policy is runtime-loaded, the backend must treat it as untrusted input. We apply strict parsing: if a `moduleLayout` fragment fails JSON parsing, the control plane rejects the request with a server error. This avoids silently serving half-valid policies that produce haunted UIs (buttons moving on their own, etc.).

4.4. **Materialization: the `/v2/me` UI contract.** The control plane returns a *materialized* profile via `/v2/me`. This response is the UI contract: it tells the web tier who the actor is, what effective tenant/role is in force (via `view-as`), what modules exist, and what layout fragments should shape the interface.

Crucially, `/v2/me` returns both:

- `actor` — immutable identity (who is authenticated),

- `view_as` — effective context (who the actor is operating *as*).

```
{
  "actor": {
    "name": "Ada Lovelace",
    "email": "ada@acme.example",
    "tenant_id": "causify",
    "user_role": "admin",
    "groups": ["causify:admin", "acme:member"]
  },
  "view_as": {
    "view_as_self": false,
    "tenant_id": "acme",
    "user_role": "member",
    "modules": ["horizon", "grid"],
    "module_layout": {
      "common": {
        "sideBar": { "defaultCollapsed": false, "showAdmin": true },
        "feedback": { "enabled": true },
        "topBar": { "enabled": true, "title": "Horizon" }
      },
      "horizon": {
        "enabled": true,
        "pageTitle": "Inventory Management",
        "dashboard": { "enabled": true },
        "gridView": { "sidePanel": { "enabled": true } }
      }
    }
  },
  "impersonatable": [
    { "tenant_id": "acme", "roles": ["demo", "member", "admin"] },
    { "tenant_id": "xyz", "roles": ["member"] }
  ]
}
```

**Listing 3.** Simplified `/v2/me` response. The `view_as` block materializes modules + layout for the effective tenant/role.

Caching. The control plane supports standard caching semantics (ETag-based conditional requests) and short TTL caching headers. This keeps the UI responsive while still enabling policy edits to take effect quickly.

Why this matters. Once `/v2/me` exists, the UI no longer needs to infer the user's world. It can simply render the world it was handed.

## 5. Runtime Composition: Making the UI Tell the Truth

This section describes how the web tier turns the UI contract into pixels without giving the browser a second job as an authorization engine.

5.1. **Abilities: one source of truth, exported for UX.** Authorization is enforced server-side, but the UI still needs to know what to show. We use CASL [5] to represent abilities and explicitly avoid duplicating rules in the frontend.

Design.

- The server builds the canonical ability from session-backed profile data (modules + role).
- The server serializes the ability rules using CASL's packed format.
- The client unpacks these rules to construct an identical ability object for UI/UX gating only.

```
import { packRules } from '@casl/ability/extra'

// GET /api/auth/user/ability
const ability = await getUserAbility()
return { rules: packRules(ability.rules) }
```

**Listing 4.** Server-authoritative abilities: rules are built server-side and packed for client use. The backend remains the enforcement point.

Two-layer protection (by design). The client uses the ability to hide inaccessible UI elements and prevent dead ends. Server components and API routes still enforce authorization. In other words: the UI is truthful, but it is not trusted.

5.2. **From modules to navigation: composition, not conditionals.** The module list in `view_as.modules` drives the UI skeleton:

- The sidebar renders only modules in the list.
- Module pages are protected server-side (redirects/404) when the user lacks `view` permission on that subject.
- Cross-module UI behaviors (top bar, settings entry points, feedback widgets) are controlled by `common` layout fragments.

This yields an important property: tenant onboarding becomes a configuration change, not a UI rewrite. The product behaves like a platform instead of a branching forest of `if (tenant == ...)`.

5.3. **Lazy module layout fetching and deep merging.** Shipping an entire tenant UI contract on every request is wasteful. Instead, we lazy-load module-specific configuration on demand. Mechanism. The web tier exposes:

`/api/auth/user/module-layout?module=<name>`

This endpoint:

(1) constructs a fallback default layout for `common` and (optionally) the requested module,
(2) checks module access via server-side CASL for non-`common` modules,
(3) merges the tenant-provided layout fragments from `/v2/me` over defaults using a recursive deep-merge.

```
function deepMerge(target, source) {
  const result = { ...target }
  for (const key in source) {
    const s = source[key]
    const t = result[key]
    if (s !== null && s !== undefined) {
      if (isObject(s) && isObject(t)) result[key] = deepMerge(t, s)
      else result[key] = s
    }
  }
  return result
}
```

**Listing 5.** Merge semantics: nested objects recursively merge, primitives/arrays replace. This enables partial overrides without forcing tenants to copy entire configs.

Why deep-merge matters. Deep-merge keeps tenant config surgical. A tenant can override `common.sideBar.defaultCollapsed` without duplicating the rest of the UI definition, keeping configuration maintainable and reviewable.

5.4. **Truthful UX as an invariant.** We treat "no broken doors" as an invariant:

- If the backend denies access to a module, the module does not appear in navigation.
- If a user tries a direct URL, server-side checks redirect/deny.
- If config asks for a module the user lacks, CASL checks prevent module config from being served.

Put differently: the UI is not permitted to hallucinate. (That is reserved for marketing.)

## 6. Safe Impersonation (`view-as`): Support Without Chaos

Impersonation is one of those features that operators love and security teams fear. OWASP explicitly calls out account impersonation risks because it can collapse accountability and enable privilege escalation [10]. We therefore design `view-as` as a controlled capability with explicit guardrails.

6.1. **Actor vs. effective context.** Every request has:

- **actor**: the authenticated user identity (tenant, role, groups),
- `view-as`: an optional effective tenant/role used for policy resolution (modules + layout) and downstream scoping.

The system returns `view_as.view_as_self` so the UI can signal when the user is impersonating. This is not cosmetic; it prevents support engineers from accidentally doing "real" operations in the wrong workspace.

6.2. **Protocol: headers, signing, and session storage.** Impersonation is conveyed via a semicolon-delimited header:

```
X-View-As:  tenant_id=<id>;role=<role>
```

The web tier generates and stores these headers server-side (session) and attaches them to control-plane calls. To add defense-in-depth, the web tier also generates an HMAC signature:

```
X-View-As-Sig:  HMAC_SHA256(secret, X-View-As)
```

The important operational detail is that the signed headers are stored on the server side of the web tier session. The browser does not hold the secret and does not directly manage impersonation headers.
Note. The control plane currently validates `view-as` using the allow-list; verifying the signature at the API tier is recommended and listed as future work (Section 10).

6.3. **Enforcement: allow-lists, not vibes.** The control plane rejects impersonation unless:

- `tenant_id` exists in `impersonatable`,
- `role` is valid for that tenant entry,
- required header keys are present.

This prevents users from "guessing" a tenant ID and role combination and hoping something accepts it. It also prevents a web tier bug from becoming a permission escalation, because the control plane is the final gate.

6.4. **Downstream scoping: tenant-aware context.** Once validated, `view-as` affects:

- **policy resolution:** modules and layout are fetched for effective tenant/role,
- **tenant scoping:** the system resolves a tenant-specific prefix (e.g., schema prefix) from the effective tenant record to scope downstream services and data access.

This last part is easy to overlook, but it is where impersonation becomes dangerous if implemented casually: `view-as` must not only change what the UI shows, it must correctly scope what the backend *touches*.

6.5. **Operational UX: visible, reversible, auditable.** A safe `view-as` workflow is more than a header:

- **Visible:** the UI clearly indicates the effective tenant/role.
- **Reversible:** users can return to self with a single action.
- **Auditable:** logs and traces attribute actions to the actor even when operating under `view-as`.

The principle is simple: `view-as` is a support tool, not a second identity. The actor remains accountable at all times.

6.6. **Recommended extensions.** We found two extensions particularly useful:

- **Read-only mode:** allow `X-View-As` to carry an explicit `mode=read-only` hint (and enforce it server-side) to prevent accidental state changes during support sessions.
- **Signature verification at the API tier:** validate `X-View-As-Sig` in the control plane for defense-in-depth.

## 7. Implementation Notes

This section grounds UI-as-Policy in concrete engineering choices. The goal is not to sell a specific stack, but to show the mechanics that make the UI *provably less prone to lying*.

7.1. **Control plane: Lambda as policy materializer.** The control plane is an AWS Lambda REST API built with Powertools for AWS Lambda (Python) [3]. The handler uses an `APIGatewayRestResolver` and exposes both `/v1/me` (legacy) and `/v2/me` (actor/`view-as` model).

Three middlewares do most of the heavy lifting:

(1) **Authentication & context middleware.** It validates the JWT, extracts `custom:tenantId`, derives the role from `cognito:groups`, computes `available_impersonations`, validates `X-View-As` (if present) against that allow-list, and resolves `schema_prefix` from the effective tenant.

(2) **ETag middleware.** It supports conditional requests via `If-None-Match` and returns `304 Not Modified` when appropriate, reducing repeated payload bytes.

(3) **Response headers middleware.** It adds security headers (CSP, HSTS, etc.) and sets a short shared cache TTL suitable for profile-like resources.

Semicolon-delimited `view-as` header. `view-as` is expressed as a semicolon-delimited header that the middleware parses into a dict, e.g. `tenant_id=acme;role=member`. This is intentionally simple: it is easy to generate, easy to log, and hard to misunderstand at 03:00.

```python
# Token -> Claims -> Tenant_Id -> User_Role
tenant_id = claims["custom:tenantId"]
user_role = extract_user_role_from_groups(tenant_id, claims.get("cognito:groups"))

available = get_available_impersonations(tenant_id, user_role, claims.get("cognito:groups"))
view_as = parse_semicolon_header(event["headers"].get("X-View-As"))
if view_as:
    assert "tenant_id" in view_as and "role" in view_as
    assert view_as["tenant_id"] in [x["tenant_id"] for x in available]
    assert view_as["role"] in roles_for(view_as["tenant_id"], available)

effective_tenant = view_as["tenant_id"] if view_as else tenant_id
claims["schema_prefix"] = get_tenant_by_id(effective_tenant).schema_prefix
event["claims"] = claims
```

**Listing 6.** Control-plane middleware enforces view-as via allow-lists and resolves effective tenant scope. (Simplified from production code.)

Policy parsing is strict. Tenant policy lives in DynamoDB and includes `moduleLayout` fragments stored as JSON strings. During materialization, each fragment is parsed; invalid JSON is treated as an error (not as "best effort"). This avoids serving half-valid policies that produce inconsistent UI composition.

7.2. **Edge security: middleware as first defense.** The web tier implements defense-in-depth via Next.js middleware. Before any server component renders, the middleware:

(1) **Validates paths:** rejects path traversal attempts (`..`, null bytes, encoded sequences).
(2) **Applies security headers:** Content-Security-Policy, HSTS (production), X-Frame-Options (DENY), X-Content-Type-Options.
(3) **Checks session:** redirects unauthenticated users to the identity provider, preserving the original path for post-login redirect.

Importantly, middleware handles *authentication* and transport-level security; *authorization* remains with the CASL-based policy model enforced in server layouts.

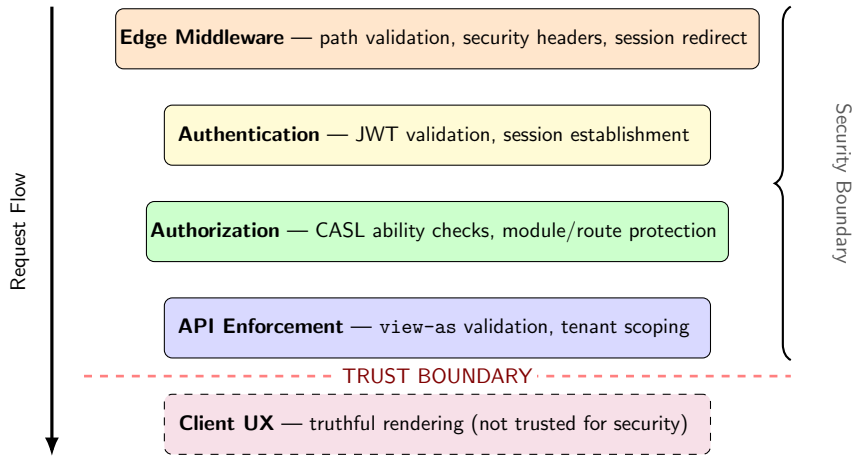


**Figure 2.** Defense-in-depth architecture. Each layer provides independent protection: Edge blocks malformed requests; Authentication validates tokens; Authorization enforces CASL permissions; API validates `view-as` and scopes tenants. Layers 1–4 form the security boundary. The Client layer uses server-exported rules for truthful UX but is never trusted for authorization.

7.3. **Web tier: Next.js as the contract consumer.** The web tier is implemented in Next.js and serves two roles: (i) it renders UI, and (ii) it mediates between the browser and the control plane so that sensitive context (like signed `view-as` headers) remains server-side.
Session storage. We use `iron-session` [12] to store encrypted session data in cookies. In practice, we keep a separation between:

- **main session:** authentication tokens and durable user profile,
- `view-as` **session:** view-as headers (and signature), stored server-side and not exposed to client JavaScript.

Server-side API client. The web tier uses a server API client that attaches:

- `Authorization: Bearer <JWT>`
- optional `X-View-As: tenant_id=...;role=...`
- optional `X-View-As-Sig: <hmac>`

to calls to the control plane. The browser never needs to construct these headers directly.

7.4. **Ability endpoint: exporting truth for UX (not for security).** The endpoint `/api/auth/user/ability` returns packed CASL rules. The server constructs the ability from the authoritative session profile and then exports it for UI gating [5]. The endpoint is explicitly non-cacheable to avoid stale UX when switching tenants/roles.

```
export const GET = withAuth(async () => {
  const packedRules = await getUserAbilityRules()
  return NextResponse.json(
    { rules: packedRules, timestamp: Date.now() },
    { headers: { 'Cache-Control': 'private, no-cache, no-store' } }
  )
})
```

**Listing 7.** Ability rules are built and packed on the server. The client uses them for truthful UX, not as a security gate.

7.5. **Module layout endpoint: lazy config with server checks.** The endpoint `/api/auth/user/module-layout` implements on-demand module config retrieval. It deep-merges tenant fragments over shipped defaults and applies a server-side CASL check before returning module-specific layouts (all modules other than `common` require `can('view', module)`).

This endpoint is where we enforce the rule:

**A module configuration is itself a privileged resource.**

If the user cannot view a module, the server will not serve its configuration, even if the client asks nicely.

7.6. **Signing `view-as`: a belt to match the suspenders.** The web tier provides `/api/auth/view-as/sign`. It validates a requested tenant/role selection against `impersonatable` (from `/v2/me`), then generates an HMAC-SHA256 signature over the `X-View-As` string and stores `viewAs` and `viewAsSig` in the server-side session.

This is defense-in-depth. The control plane still enforces allow-lists; the signature primarily protects against accidental header fabrication and makes tampering more detectable in logs.

## 8. Evaluation

We evaluate UI-as-Policy along three axes: (i) *truthfulness* (does the UI advertise only what the system will allow?), (ii) *operational leverage* (does tenant variability become configuration rather than redeploy?), and (iii) *runtime overhead* (what does materialization cost?).

8.1. **Truthfulness invariant: "no broken doors".** We treat truthful UX as a property to be tested, not hoped for.

**Definition 8.1** (Truthful UX)**.** A UI is *truthful* if, for any effective tenant/role context, every reachable UI action corresponds to a backend operation that will not be rejected solely due to authorization, under the same context.

We cannot prove this property for arbitrary UIs, but we can enforce strong sufficient conditions in Causify Platform:

**Theorem 8.2** (Sufficient conditions for truthful module navigation)**.** *Assume: (i) navigation is rendered exclusively from `view_as.modules` returned by `/v2/me`; (ii) module configuration is served only if `can('view', module)` holds server-side; and (iii) backend APIs enforce authorization independently of the client. Then the UI will not present a module entry that the backend would deny for that effective context.*

*Proof sketch.* By (i), the UI cannot render a module entry unless it is present in the server-materialized module list for the effective context. By (ii), even if a client attempts to force-load configuration for an unauthorized module, the web tier denies it. Finally, by (iii), even a compromised UI cannot bypass backend authorization. Together these eliminate the most common drift path: UI showing modules the backend will not honor. □

Practical test harness. We recommend an automated "door check": for each tenant/role pair, render the sidebar from `/v2/me`, visit each module route, and confirm that: (a) the route renders (or redirects) consistently with permissions, and (b) module-layout requests are denied for unauthorized modules. This can be implemented as integration tests in CI and is especially valuable after policy migrations.

8.2. **Operational leverage: configuration over redeploy.** UI-as-Policy turns tenant onboarding into a data operation:

- Add tenant record in DynamoDB (e.g., `group.<role>.modules` and `.moduleLayout` fields).
- Add Cognito groups following the `tenant:role` convention.
- Users immediately receive a correct UI contract via `/v2/me`.

The measurable KPI is not philosophical; it is mundane: *how often does onboarding require a code change?* We recommend tracking: (1) number of deployments triggered by tenant enablement or layout changes, and (2) mean time from "tenant policy edit" to "tenant UI reflects change". ETag support reduces repeated fetch cost while preserving rapid propagation.

8.3. **Runtime overhead: materialization and caching.** The control plane sets a short cache TTL and ETag headers for `/v2/me`, enabling clients to revalidate cheaply (304 responses). Materialization cost is dominated by: (i) JWT decoding and role resolution, (ii) DynamoDB read for tenant policy, (iii) JSON parsing for layout fragments.

We recommend reporting:

- P50/P95 latency for `/v2/me` (cold vs warm),
- average response size and 304 hit rate (conditional requests),
- number of JSON fragments parsed per request (proxy for config complexity).

What we do not claim. We do not claim UI-as-Policy eliminates all authorization bugs. It does reduce a specific, expensive class: frontend/backend drift and the operational churn it causes.

8.4. **Policy-driven UI footprint: how much of the codebase obeys policy?** A useful question is: what fraction of the UI is actually derived from policy, and how large is the policy surface relative to the application? We measure the production codebase.

| Component | Size |
|---|---|
| Total TypeScript/TSX files | 881 files |
| Total lines of code | ∼187,800 |
| React components | 225 |
| UI primitive components (Radix-based) | 52 |
| Domain-specific components | 170+ |
| Custom React hooks | 33 |
| Next.js page routes | 27 |
| Next.js layout files | 17 |
| *Policy-driven rendering:* | |
| Components using config/module gating | 48 |
| Modules defined in module mapping | 7 |
| CASL permission subjects | 14+ default features |
| Sidebar modes supported via config | 6 |

**Table 3.** UI codebase footprint. 48 of 225 components (∼21%) directly consume policy inputs (module flags, layout config, or CASL abilities) to determine what to render. The remaining components are presentation-only and inherit policy decisions from their parents.

8.5. **Authorization layers: defense-in-depth accounting.** UI-as-Policy does not rely on a single enforcement point. We count the distinct layers a request traverses and the code that implements each.

| Layer | Enforcement | Mechanism |
|---|---|---|
| Edge middleware | Server | Next.js middleware: session check, security headers, path routing |
| Server components | Server | `getSession()` in server components, redirect on missing auth |
| CASL abilities | Server + Client | Built server-side from `/v2/me`, packed and exported to client for UX |
| Module gating | Server | `can('view', module)` checked before serving layout config |
| View-as signing | Server | HMAC-signed `X-View-As-Sig` header, browser never holds secret |
| Control plane | Server | Lambda middleware validates JWT, view-as allow-list, schema prefix |

**Table 4.** Authorization layers in the UI-as-Policy stack. Six layers enforce policy between the browser and the data plane. The client uses CASL for UX consistency but is never trusted for security.

8.6. **Module gating: how much surface does policy remove?** The platform defines 7 modules (Horizon, Grid, Optima, Panorama, Frontier, Sentinel, DataMap). Navigation, page routes, and layout fragments are rendered only for enabled modules. We quantify the reduction.

| Tenant profile | Visible modules | Reduction |
|---|---|---|
| Full platform (all 7 modules) | 7 | — |
| Energy tenant (Grid only) | 1 | ∼86% |
| Supply chain tenant (Horizon only) | 1 | ∼86% |
| Two-module tenant | 2 | ∼71% |

**Table 5.** Module visibility reduction. A single-module tenant sees one sidebar entry, one dashboard card, and one set of pages out of seven. Routes, layout fragments, and API clients for disabled modules are never loaded.

Combined with the control plane's OpenAPI filtering (which removes 67 to 90% of API endpoints for limited-module tenants), this means a single-module tenant sees a UI and API surface that is a small fraction of the full platform.

8.7. **Configuration surface: what is policy-as-data?** The paper claims UI structure comes from data, not code. We inventory the policy surface stored in DynamoDB and consumed at runtime.

| Config element | Count | What it controls |
|---|---|---|
| Module list per role | per tenant/role | Which sidebar entries, dashboard cards, and routes appear |
| `moduleLayout` fields | per tenant/role | TopBar (title, search, notifications, theme, help), SideBar (mode, profile, collapsibility), feedback, settings tabs |
| Sidebar modes | 6 variants | hover, classic, auto-collapse, always-expanded, mini, drawer |
| Sub-module access | per module | Fine-grained feature access within a module (e.g., `horizon:supply-chain`) |
| Module aliases | per tenant | Backend URL overrides for white-label deployments |

**Table 6.** Policy-as-data surface. Every element listed is stored in DynamoDB and consumed at runtime via `/v2/me` and the module-layout endpoint. Changes take effect without code deployment.

8.8. **Policy coverage: what percentage of the UI is policy-driven?** To measure how deeply policy penetrates the UI, we count every surface element and classify it as *always visible*, *policy-gated* (shown/hidden by module or role), or *policy-configured* (behavior controlled by `moduleLayout`).

| UI surface | Total | Policy-driven | Coverage |
|---|---|---|---|
| Sidebar navigation items | 11 | 8 | 73% |
| Dashboard module cards | 7 | 7 | 100% |
| Page routes | 22 | 8 | 36% |
| Settings tabs | 6 | 5 | 83% |
| CASL permission subjects | 25+ | 25+ | 100% |
| `moduleLayout` config fields | 150+ | 150+ | 100% |

**Table 7.** Policy coverage across UI surfaces. High-value surfaces (navigation, dashboard, settings, layout configuration) are overwhelmingly policy-driven. Page routes have lower coverage because generic pages (profile, docs, what's new) are always accessible.

Permission model depth. The CASL ability model defines 3 actions (`view`, `manage`, `use`) across 25+ subjects: 7 modules, 5 generic pages, 16 feature permissions (`feature:export`, `feature:charts`, `feature:notifications`, etc.), sub-module subjects (e.g., `horizon:supply-chain`), and config-driven subjects derived from `moduleLayout` paths (e.g., `settings:tabs:data-connectors:enabled`). The 150+ `moduleLayout` fields control everything from sidebar mode (6 variants) to individual chart visibility, KPI card layout, and per-tab settings access.

8.9. **Interpretation.** The quantitative results confirm the three evaluation axes. The truthfulness invariant is supported by 6 defense layers and a permission model (CASL) that is built server-side and exported to the client (never the reverse). Operational leverage is confirmed by the policy-as-data surface: module lists, layout fragments, sidebar modes, and sub-module access are all DynamoDB records that take effect without deployment. Runtime composition is validated by the codebase structure: 61% of discrete UI surface elements (28 of 46 across navigation, dashboard, routes, and settings) are policy-driven, single-module tenants see an 86% reduction in visible modules, and high-value surfaces (navigation, dashboard, settings) have 73 to 100% policy coverage with 150+ configurable layout fields.

The UI is not merely "gated." It is *composed from policy*, and the numbers show how much of the surface that policy controls.

## 9. Related Work

ABAC, RBAC, and policy expressiveness. ABAC formalizes authorization decisions based on attributes rather than static roles and is a standard reference point [8]. UI-as-Policy is compatible with ABAC: the server may compute abilities using attributes, but the key contribution here is the UI contract that prevents the frontend from guessing.

Feature toggles and product variability. Feature flags separate release from deployment and are widely used for rollout control [7]. However, feature flags alone do not guarantee UI truthfulness: they can become a parallel universe of conditions that diverge from authorization. UI-as-Policy can be viewed as feature toggles with teeth: configuration is scoped by tenant/role and validated against the same ability model.

Policy engines and external authorization services. OPA provides a general-purpose policy engine and a declarative language to offload authorization decisions [9]. Cedar (used by Amazon Verified Permissions) provides a language for authorization policies and standardized evaluation models [4, 6]. These systems are highly relevant when the goal is centralized authorization logic. UI-as-Policy is orthogonal: we focus on how policy decisions become UI structure in a multi-tenant product.

Relationship-based authorization and global systems. Zanzibar describes a global, consistent authorization system used across Google services [11]. While Causify Platform does not implement a Zanzibar-style relationship store, Zanzibar underscores an important theme: authorization becomes a distributed-systems problem at scale. UI-as-Policy addresses a nearby problem: when authorization scales, the UI must not become an unreliable narrator.

Impersonation and operational risk. OWASP explicitly highlights account impersonation as a security risk in citizen development contexts [10]. Our `view-as` design aligns with the recommended mitigations: explicit actor attribution, guardrails, and defense-in-depth.

## 10. Limitations and Future Work

UI-as-Policy makes a specific trade: it shifts some product definition into data. This buys flexibility, but it introduces new responsibilities.

Policy schema validation. Today, malformed `moduleLayout` JSON is rejected at runtime. Future work should include:

- a versioned JSON schema for layout fragments,
- offline validation tooling (CI + pre-commit),
- migration helpers when layout structures evolve.

Signature verification in the control plane. The web tier signs `X-View-As`. Verifying `X-View-As-Sig` in the control plane would strengthen defense-in-depth and make header provenance auditable end-to-end.

Push-based invalidation. ETags and short TTL caching work well for most edits, but critical policy updates may benefit from push invalidation (e.g., event-driven cache busting or policy version bumps in the session).

Finer-grained capabilities. Today, capabilities are primarily module-centric. Extending to feature- and resource-level controls (within a module) is feasible, but must preserve the single-source-of-truth property and avoid reintroducing drift through ad-hoc UI conditions.

Governance and authoring experience. A policy-as-data system needs governance: who can change tenant policy, how changes are reviewed, and how rollbacks work. A structured policy editor (with preview and diffing) is a natural next step.

## 11. Conclusion

Multi-tenant SaaS systems often fail in a deeply human way: the backend tells the truth, and the UI tells a story. The story is usually optimistic. Sometimes it is dangerously optimistic.

UI-as-Policy replaces optimism with a contract. By materializing a tenant/role UI contract (UI-as-Data) server-side and deriving both UX and authorization from a unified ability model, Causify Platform reduces frontend/backend drift, turns tenant variability into configuration, and enables safe `view-as` workflows without shared admin accounts.

The headline is simple:

**If the UI can lie about access, it is part of the security boundary.**

Once you accept that, the rest becomes engineering.

## References


[1] AWS Cognito Documentation, *Amazon cognito user pools*. Accessed 2026-01-18.

[2] AWS Cognito Groups Documentation, *Adding groups to a user pool (amazon cognito)*. Accessed 2026-01-18.

[3] AWS Lambda Powertools, *Powertools for aws lambda (python)*. Accessed 2026-01-18.

[4] AWS Verified Permissions, *Amazon verified permissions documentation*. Accessed 2026-01-18.

[5] CASL Authors, *Casl: Isomorphic authorization javascript library*. Accessed 2026-01-18.

[6] Cedar Policy Language Authors, *Cedar policy language reference guide*. Accessed 2026-01-18.

[7] Martin Fowler, *Feature toggles (aka feature flags)*. Accessed 2026-01-18.

[8] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone, *Guide to attribute based access control (abac) definition and considerations*, Technical Report Special Publication 800-162, National Institute of Standards and Technology (NIST), 2014. Includes editorial/substantive updates through 2019.

[9] Open Policy Agent Contributors, *Open policy agent (opa) documentation*. Accessed 2026-01-18.

[10] OWASP Foundation, *Cd-sec-02: Account impersonation*, 2022. Accessed 2026-01-18.

[11] Ruoming Pang, Manuel Cáceres, et al., *Zanzibar: Google's consistent, global authorization system*, Proceedings of the usenix annual technical conference (atc), 2019. Accessed 2026-01-18.

[12] vvo and contributors, *iron-session: secure, stateless, and cookie-based sessions*. Accessed 2026-01-18.